

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении лабораторных работ
По дисциплине: «Методы проектирования баз данных»

Выполнил студент
группы 5130201/40002

_____ Семенов И. А.

Проверил
преподаватель

_____ Попов С. Г.

«_____» _____ 2026г.

Содержание

Введение	2
1 Постановка задачи	3
2 Основная часть	4
2.1 Лабораторная работа № 1	4
2.1.1 Задача	4
2.1.2 Создание представления	4
2.1.3 Проверка представления	5
2.1.4 Запрос с условием	7
2.1.5 Охват форматов по категориям	8
2.2 Лабораторная работа № 2	10
2.2.1 Задача	10
2.2.2 Реализация триггеров	10
2.2.3 Проверка работы триггеров	14
2.2.4 Схема базы данных	16
3 Заключение	18

Введение

1 Постановка задачи

2 Основная часть

2.1 Лабораторная работа № 1

2.1.1 Задача

В ходе лабораторной работы необходимо:

1. создать представление, содержащее результат группировки данных с двумя агрегатными функциями `COUNT`;
2. составить простой запрос к созданному представлению с условием `WHERE`;
3. составить запрос, в котором представление используется совместно с операциями соединения `JOIN`.

2.1.2 Создание представления

Необходимо создать представление, которое для каждого начертания хранит число поддерживаемых форматов и количество градаций насыщенности.

Одно начертание может храниться в таблице `Typeface` в нескольких форматах и градациях насыщенности. Чтобы не выполнять одинаковый подсчёт в каждом запросе, создаётся отдельное представление.

В нём для каждого начертания хранятся название, идентификатор гарнитуры, число форматов и число градаций насыщенности. Определение представления приведено в листинге 1.

В формулах используются обозначения: T — таблица `Typeface`, F — таблица `Format`, W — таблица `Weight`, FF — таблица `Font_family`, C — таблица `Category`, V — созданное представление. Символ $\bowtie_k$ обозначает соединение по полю k , σ — выборку, π — проекцию, γ — группировку с агрегированием.

Листинг 1: Создание представления

```
1 CREATE OR REPLACE VIEW typeface_format_weight_stats_view AS
2 SELECT
3     t.id_font_family,
4     t.name AS typeface_name,
5     COUNT(DISTINCT f.id_format) AS format_count,
6     COUNT(DISTINCT w.id_weight) AS weight_count
7 FROM typeface AS t
8 JOIN format AS f ON f.id_format = t.id_format
9 JOIN weight AS w ON w.id_weight = t.id_weight
10 GROUP BY
11     t.id_font_family,
12     t.name;
```

Агрегатная функция `COUNT(DISTINCT ...)` подсчитывает количество различных значений. Поэтому повторяющиеся форматы и градации насыщенности учитываются только один раз. Для получения фактических форматов и градаций насыщенности таблица `Typeface` соединяется с таблицами `Format` и `Weight`.

Создание представления описывается формулой

$$X = (T \bowtie_{id_format} F) \bowtie_{id_weight} W,$$

$$V = \gamma_{\substack{id_font_family, name; \\ count_distinct(id_format) \rightarrow format_count, \\ count_distinct(id_weight) \rightarrow weight_count}}(X).$$

2.1.3 Проверка представления

Текст запроса: Вывести первые десять начертаний из созданного представления вместе с числом поддерживаемых форматов и градаций насыщенности.

Запрос приведён в листинге 2.

Листинг 2: Проверка содержимого представления

```

1 SELECT
2     v.id_font_family,
3     v.typeface_name,
4     v.format_count,
5     v.weight_count
6 FROM typeface_format_weight_stats_view AS v
7 ORDER BY v.id_font_family, v.typeface_name
8 LIMIT 10;
```

Реляционная формула имеет вид

$$R_1 = \pi_{id_font_family, typeface_name, \substack{format_count, weight_count}}(V).$$

План и результат выполнения показаны на рисунках 1 и 2.

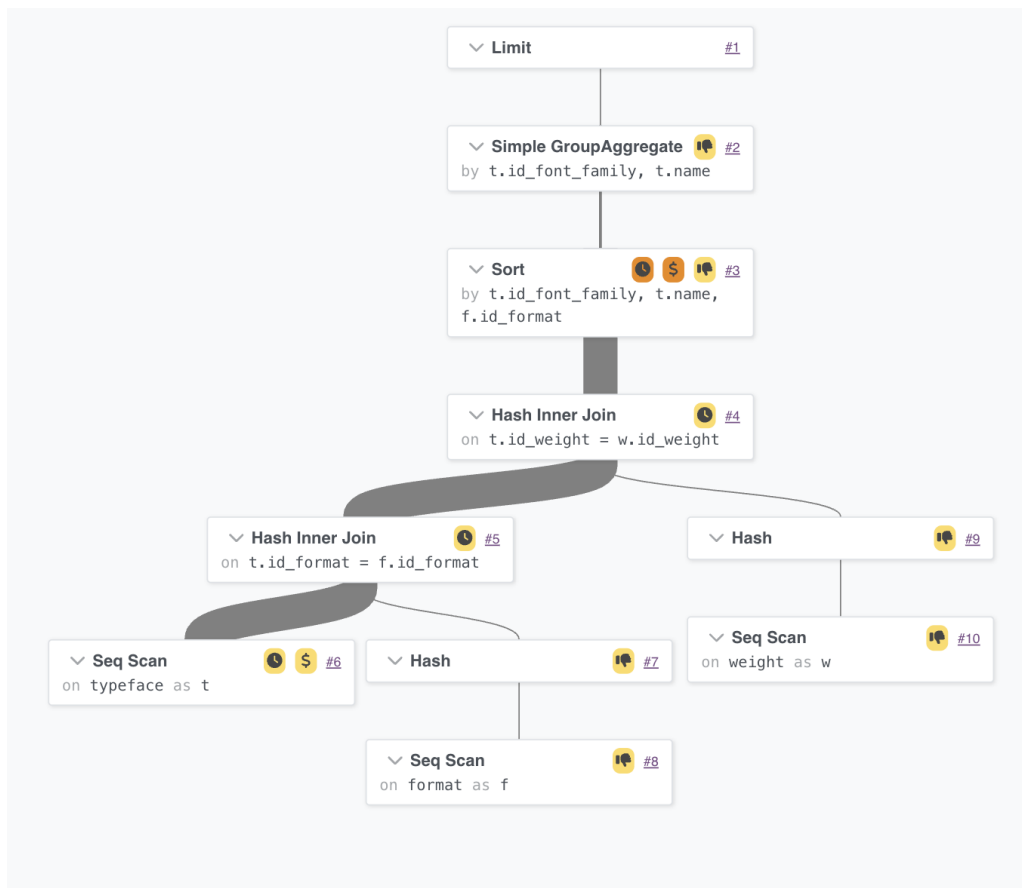


Рис. 1: План выполнения запроса 1

PostgreSQL читает таблицы `Typeface`, `Format` и `Weight`, затем соединяет их по полям `id_format` и `id_weight`. Полученные строки сортируются по полям группировки, после чего узел `GroupAggregate` рассчитывает количество различных форматов и градаций насыщенности каждого начертания.

cw-sem4 - zsh

✓ PostgreSQL: run current f...

/dev/fd/12:25: command not found: compdef

id_font_family	typeface_name	format_count	weight_count
1	Typeface 1	1	5
2	Typeface 2	1	2
3	Typeface 3	1	9
4	Typeface 4	1	7
5	Typeface 5	1	5
6	Typeface 6	1	8
7	Typeface 7	1	7
8	Typeface 8	1	9
9	Typeface 9	1	5
10	Typeface 10	1	2

(10 rows)

Рис. 2: Результат выполнения запроса 1

2.1.4 Запрос с условием

Текст запроса: Найти число начертаний, у которых количество поддерживаемых форматов больше A .

В запросе значение A принято равным 2. Сначала условие `format_count > 2` оставляет подходящие начертания, затем `COUNT` подсчитывает их. Запрос приведён в листинге 3.

Листинг 3: Подсчёт начертаний с числом форматов больше A

```
1 SELECT
2     COUNT(v.id_font_family)
3 FROM typeface_format_weight_stats_view AS v
4 WHERE v.format_count > 2;
```

Запрос описывается формулой

$$R_2 = \gamma_{\text{count}(\text{id_font_family}) \rightarrow \text{typeface_count}} (\sigma_{\text{format_count} > A}(V)) .$$

План и результат выполнения показаны на рисунках 3 и 4.

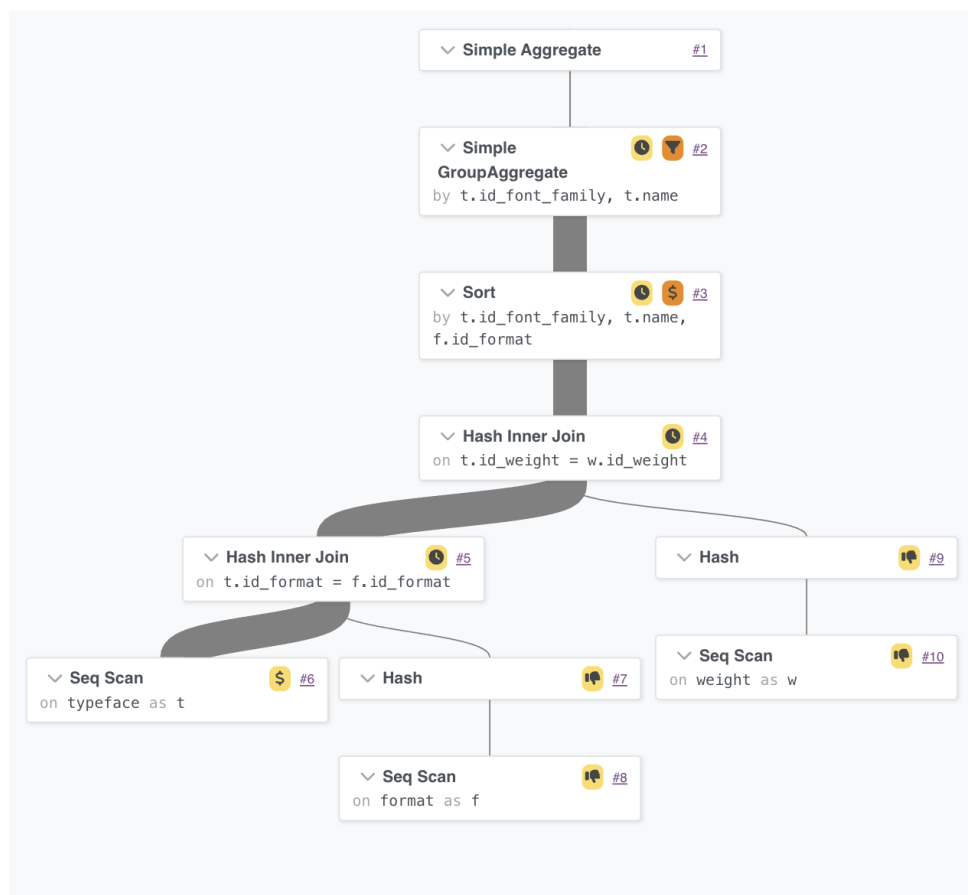


Рис. 3: План выполнения запроса 2

Сначала PostgreSQL соединяет таблицу начертаний с таблицами форматов и насыщенностей, после чего вычисляет строки представления с

помощью сортировки и `GroupAggregate`. Во время агрегирования отбрасываются начертания, у которых число форматов не больше A . Затем узел `Aggregate` подсчитывает оставшиеся строки.



```

> cw-sem4 - zsh      ✓ PostgreSQL: run current f...
/dev/fd/12:25: command not found: compdef
count
-----
      39
(1 row)

```

Рис. 4: Результат выполнения запроса 2

Условию соответствует 39 начертаний.

2.1.5 Охват форматов по категориям

Текст запроса: Найти пять категорий с наибольшим суммарным охватом поддерживаемых форматов.

Представление содержит идентификатор гарнитуры, поэтому его можно соединить с таблицами `Font_family` и `Category`. После соединения значения `format_count` суммируются отдельно для каждой категории. Запрос приведён в листинге 4.

Листинг 4: Первые пять категорий по охвату поддерживаемых форматов

```

1 SELECT
2     c.id_category,
3     c.name,
4     SUM(v.format_count) AS format_count
5 FROM typeface_format_weight_stats_view AS v
6 JOIN font_family AS ff
7     ON ff.id_font_family = v.id_font_family
8 JOIN category AS c
9     ON c.id_category = ff.id_category
10 GROUP BY c.id_category, c.name
11 ORDER BY format_count DESC, c.name
12 LIMIT 5;

```

Агрегирование по категориям и выбор первых пяти строк описываются формулами

$$\begin{aligned}
 X &= (V \bowtie_{id_font_family} FF) \bowtie_{id_category} C, \\
 Q &= \gamma_{\substack{id_category, name; \\ sum(format_count) \rightarrow format_count}}(X), \\
 R_3 &= top_5^{format_count \downarrow, name \uparrow}(Q).
 \end{aligned}$$

Представление уже содержит число форматов каждого начертания, поэтому повторно считать их в запросе не требуется. План и результат выполнения показаны на рисунках 5 и 6.

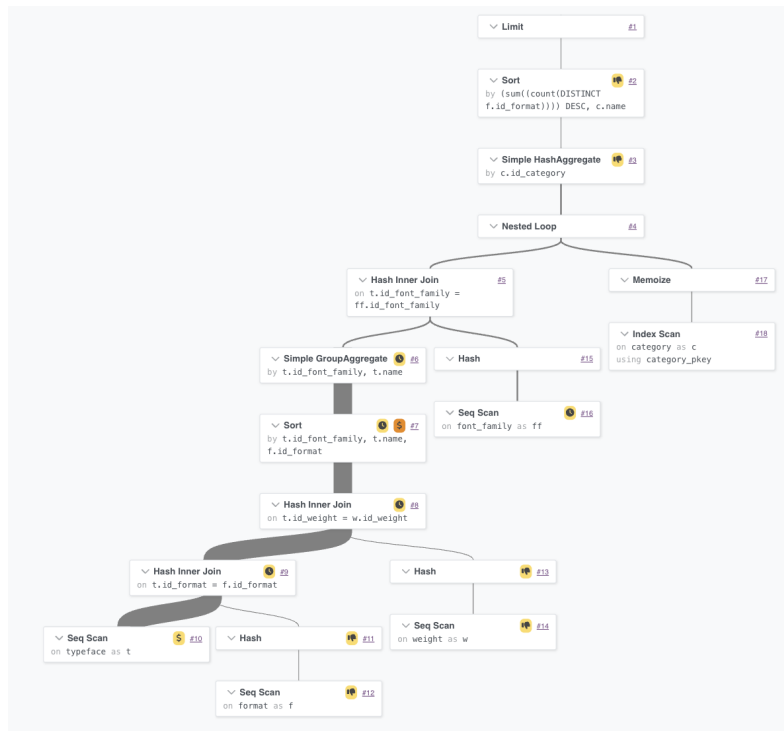


Рис. 5: План выполнения запроса 3

PostgreSQL вычисляет представление, соединяя начертания с форматами и насыщенностями. Полученные строки дополнительно соединяются с гарнитурами и категориями. Затем значения `format_count` суммируются по категориям. Завершающие сортировка и ограничение оставляют пять категорий с наибольшим охватом.

Рис. 6: Результат выполнения запроса 3

После группировки категории сортируются по убыванию охвата. Оператор `LIMIT` 5 оставляет первые пять строк. Наибольший охват в полученном результате имеет категория `Neo-grotesque`.

2.2 Лабораторная работа № 2

2.2.1 Задача

Необходимо реализовать таблицу `Typeface_format_weight_stats`, содержимое которой повторяет результат представления, созданного в первой лабораторной работе. Таблица должна хранить для каждого начертания идентификатор гарнитуры, название начертания, число поддерживаемых форматов и количество градаций насыщенности.

Актуальность агрегированных данных необходимо обеспечить с помощью триггеров на таблицах `Font_family` и `Typeface`. При изменении исходных данных соответствующие строки таблицы статистики должны обновляться автоматически.

Требуется реализовать пять триггеров, покрывающих следующие изменения:

1. изменение идентификатора гарнитуры в таблице `Font_family`;
2. удаление гарнитуры из таблицы `Font_family`;
3. добавление начертания в таблицу `Typeface`;
4. изменение гарнитуры, названия, формата или насыщенности начертания в таблице `Typeface`;
5. удаление начертания из таблицы `Typeface`.

Обновлённая схема базы данных представлена на рисунке [12](#).

2.2.2 Реализация триггеров

В PostgreSQL обработчик триггера оформляется в виде функции, возвращающей значение типа `TRIGGER`. После объявления функции команда `CREATE TRIGGER` связывает её с таблицей, событием и моментом выполнения.

Триггер `font_family_updated`. Триггер выполняется перед изменением идентификатора гарнитуры. Он переносит на новый идентификатор строки таблицы статистики и связанные начертания. Внешний ключ `typeface.id_font_family` объявляется отложенным, поэтому его проверка производится после завершения изменения гарнитуры. Реализация триггера приведена в листинге [5](#).

Листинг 5: Триггер изменения гарнитуры

```
1 ALTER TABLE typeface
2 ALTER CONSTRAINT typeface_id_font_family_fkey
```

```

3 DEFERRABLE INITIALLY DEFERRED;
4
5 CREATE OR REPLACE FUNCTION font_family_updated_fn()
6 RETURNS TRIGGER
7 LANGUAGE plpgsql
8 AS $$
9 BEGIN
10     UPDATE typeface_format_weight_stats
11     SET id_font_family = NEW.id_font_family
12     WHERE id_font_family = OLD.id_font_family;
13
14     UPDATE typeface
15     SET id_font_family = NEW.id_font_family
16     WHERE id_font_family = OLD.id_font_family;
17
18     RETURN NEW;
19 END;
20 $$;
21
22 CREATE TRIGGER font_family_updated
23 BEFORE UPDATE OF id_font_family ON font_family
24 FOR EACH ROW
25 EXECUTE FUNCTION font_family_updated_fn();

```

Триггер font_family_deleted. Триггер выполняется перед удалением гарнитуры. Сначала он удаляет принадлежащие ей начертания, после чего удаляет соответствующие агрегированные строки. Это позволяет выполнить операцию без каскадного удаления на уровне внешнего ключа. Реализация приведена в листинге 6.

Листинг 6: Триггер удаления гарнитуры

```

1 CREATE OR REPLACE FUNCTION font_family_deleted_fn()
2 RETURNS TRIGGER
3 LANGUAGE plpgsql
4 AS $$
5 BEGIN
6     DELETE FROM typeface
7     WHERE id_font_family = OLD.id_font_family;
8
9     DELETE FROM typeface_format_weight_stats
10    WHERE id_font_family = OLD.id_font_family;
11
12    RETURN OLD;
13 END;
14 $$;
15
16 CREATE TRIGGER font_family_deleted
17 BEFORE DELETE ON font_family
18 FOR EACH ROW
19 EXECUTE FUNCTION font_family_deleted_fn();

```

Триггер `typeface_inserted`. После добавления начертания триггер подсчитывает число различных форматов и градаций насыщенности для соответствующей пары из идентификатора гарнитуры и названия начертания. Если агрегированная строка уже существует, её значения обновляются конструкцией `ON CONFLICT`. Реализация приведена в листинге 7.

Листинг 7: Триггер добавления начертания

```
1 CREATE OR REPLACE FUNCTION typeface_inserted_fn()
2 RETURNS TRIGGER
3 LANGUAGE plpgsql
4 AS $$
5 BEGIN
6     INSERT INTO typeface_format_weight_stats
7     SELECT
8         t.id_font_family,
9         t.name,
10        COUNT(DISTINCT f.id_format),
11        COUNT(DISTINCT w.id_weight)
12 FROM typeface AS t
13 JOIN format AS f ON f.id_format = t.id_format
14 JOIN weight AS w ON w.id_weight = t.id_weight
15 WHERE t.id_font_family = NEW.id_font_family
16        AND t.name = NEW.name
17 GROUP BY
18     t.id_font_family,
19     t.name
20 ON CONFLICT (id_font_family, typeface_name)
21 DO UPDATE SET
22     format_count = EXCLUDED.format_count,
23     weight_count = EXCLUDED.weight_count;
24
25 RETURN NEW;
26 END;
27 $$;
28
29 CREATE TRIGGER typeface_inserted
30 AFTER INSERT ON typeface
31 FOR EACH ROW
32 EXECUTE FUNCTION typeface_inserted_fn();
```

Триггер `typeface_updated`. Триггер запускается после изменения гарнитуры, названия, формата или насыщенности начертания. Старая агрегированная строка удаляется, затем по актуальным данным заново рассчитываются показатели старой и новой групп. Реализация приведена в листинге 8.

Листинг 8: Триггер изменения начертания

```
1 CREATE OR REPLACE FUNCTION typeface_updated_fn()
```

```

2 RETURNS TRIGGER
3 LANGUAGE plpgsql
4 AS $$
5 BEGIN
6     DELETE FROM typeface_format_weight_stats
7     WHERE id_font_family = OLD.id_font_family
8         AND typeface_name = OLD.name;
9
10    INSERT INTO typeface_format_weight_stats
11    SELECT
12        t.id_font_family,
13        t.name,
14        COUNT(DISTINCT f.id_format),
15        COUNT(DISTINCT w.id_weight)
16    FROM typeface AS t
17    JOIN format AS f ON f.id_format = t.id_format
18    JOIN weight AS w ON w.id_weight = t.id_weight
19    WHERE (t.id_font_family = OLD.id_font_family AND t.name = OLD.name)
20        OR (t.id_font_family = NEW.id_font_family AND t.name = NEW.name)
21    GROUP BY
22        t.id_font_family,
23        t.name
24    ON CONFLICT (id_font_family, typeface_name)
25    DO UPDATE SET
26        format_count = EXCLUDED.format_count,
27        weight_count = EXCLUDED.weight_count;
28
29    RETURN NEW;
30 END;
31 $$;
32
33 CREATE TRIGGER typeface_updated
34 AFTER UPDATE OF id_font_family, name, id_format, id_weight ON typeface
35 FOR EACH ROW
36 EXECUTE FUNCTION typeface_updated_fn();

```

Триггер typeface_deleted. После удаления начертания триггер удаляет прежнюю агрегированную строку и повторно рассчитывает её по оставшимся начертаниям. Если записей с такой парой гарнитуры и названия больше нет, новая строка не создаётся. Реализация приведена в листинге 9.

Листинг 9: Триггер удаления начертания

```

1 CREATE OR REPLACE FUNCTION typeface_deleted_fn()
2 RETURNS TRIGGER
3 LANGUAGE plpgsql
4 AS $$
5 BEGIN
6     DELETE FROM typeface_format_weight_stats
7     WHERE id_font_family = OLD.id_font_family

```

```

8      AND typeface_name = OLD.name;
9
10     INSERT INTO typeface_format_weight_stats
11     SELECT
12         t.id_font_family,
13         t.name,
14         COUNT(DISTINCT f.id_format),
15         COUNT(DISTINCT w.id_weight)
16     FROM typeface AS t
17     JOIN format AS f ON f.id_format = t.id_format
18     JOIN weight AS w ON w.id_weight = t.id_weight
19     WHERE t.id_font_family = OLD.id_font_family
20         AND t.name = OLD.name
21     GROUP BY
22         t.id_font_family,
23         t.name;
24
25     RETURN OLD;
26 END;
27 $$;
28
29 CREATE TRIGGER typeface_deleted
30 AFTER DELETE ON typeface
31 FOR EACH ROW
32 EXECUTE FUNCTION typeface_deleted_fn();

```

2.2.3 Проверка работы триггеров

Для каждого триггера подготовлен отдельный тестовый запрос. Все проверки выполняются внутри транзакции, которая завершается командой `ROLLBACK`; поэтому тестовые изменения не сохраняются в базе данных. Рассмотренные сценарии приведены в таблице 1.

Таблица 1: Примеры работы триггеров

№	Действие
1	Изменение идентификатора гарнитуры (UPDATE id_font_family в Font_family). Идентификатор изменяется в связанных начертаниях и строках Typeface_format_weight_stats.
2	Удаление гарнитуры (DELETE из Font_family). Связанные начертания и агрегированные строки удаляются триггером.
3	Добавление начертания (INSERT в Typeface). В таблице статистики создаётся новая строка либо пересчитываются существующие счётчики.
4	Изменение начертания (UPDATE полей id_font_family, name, id_format или id_weight). Показатели старой и новой группы пересчитываются.
5	Удаление начертания (DELETE из Typeface). Счётчики группы пересчитываются, а при отсутствии других начертаний агрегированная строка удаляется.

Результаты выполнения тестовых запросов представлены на рисунках 7–11.

```
UPDATE 1
 id_font_family | typeface_name | format_count | weight_count
-----+-----+-----+-----
          10000 | Typeface 1    |             1 |             5
(1 row)
```

Рис. 7: Результат изменения идентификатора гарнитуры

```
INSERT 0 1
INSERT 0 1
DELETE 1
 count
-----
          0
(1 row)
```

Рис. 8: Результат удаления гарнитуры

```

INSERT 0 1
 id_font_family | typeface_name | format_count | weight_count
-----+-----+-----+-----
          1 | Trigger insert test |          1 |          1
(1 row)

```

Рис. 9: Результат добавления начертания

```

UPDATE 1
 id_font_family | typeface_name | format_count | weight_count
-----+-----+-----+-----
          1 | Trigger update test |          1 |          1
(1 row)

```

Рис. 10: Результат изменения начертания

```

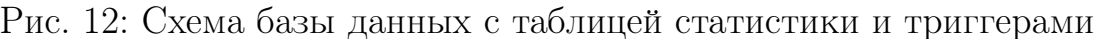
INSERT 0 1
DELETE 1
count
-----
      0

```

Рис. 11: Результат удаления начертания

2.2.4 Схема базы данных

В исходную схему добавлена таблица `Typeface_format_weight_stats`. Пунктирные стрелки показывают операции, при выполнении которых триггеры изменяют содержащиеся в ней агрегированные значения. Обновлённая схема представлена на рисунке [12](#).



3 Заключение